Introduction to Reinforcement Learning
– Term Project –

서강대학교 기계공학과

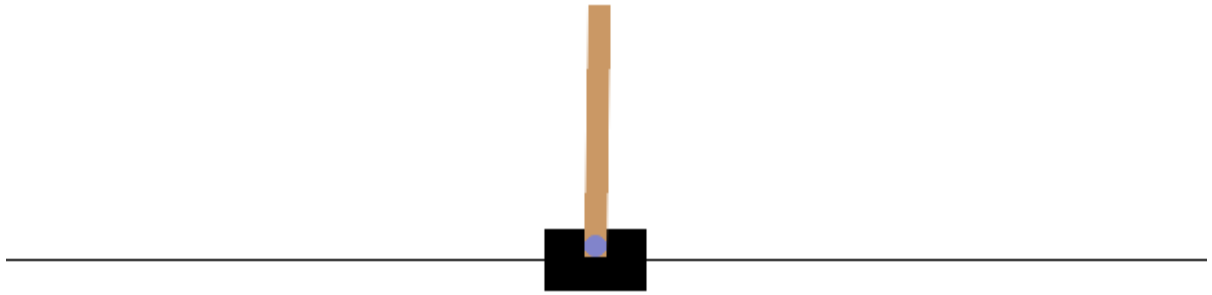
기말쉽게내조

320250213 표진우

120250367 이충현

Github page: https://github.com/RyanPyo/MPC-RL-cartpole

Contents



Introduction

<Project Background>

- In real robotic control systems, model uncertainty significantly degrades the performance of model-based controllers.
- Reinforcement Learning (RL) can compensate for uncertainty and nonlinearities through learning, but it often suffers from stability issues and low sample efficiency.
- Recently, hybrid control frameworks that combine the stability and constraint-handling capability of Model Predictive Control (MPC) with the adaptability of RL have gained considerable attention.

<Problem Definition>

- By adding an RL agent's residual action $u_{residual}$ to the MPC controller's output, it is possible to compensate for model mismatch.
- This project systematically compares pure MPC control with MPC + Residual RL control in terms of the following evaluation items

  1) Stability under mass variation 2) Stability under push disturbances

  3) Stability under model mismatch + push disturbance

System Modeling

1. Cart Pole system modeling

- State vector = $\{x, \dot{x}, \theta, \dot{\theta}\}$
- State space equation of cart pole

$$\dot{X} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & -\frac{(I+ml^2)}{\Psi} & \frac{m^2l^2g}{\Psi} & 0 \\ 0 & 0 & 0 & 1 \\ 0 & -\frac{mlb}{\Psi} & \frac{mgl(M+m)}{\Psi} & 0 \end{bmatrix} X + \begin{bmatrix} 0 \\ \frac{I+ml^2}{\Psi} \\ 0 \\ \frac{ml}{\Psi} \end{bmatrix} u$$

$$\Psi = I(M+m) + mMl^2$$

Eq. 1: Linearized state-space model of the cart–pole dynamics.

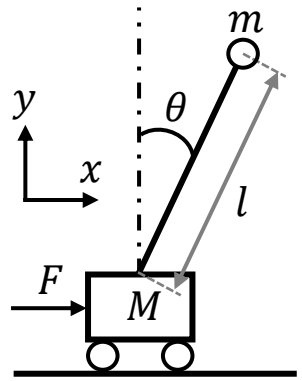


Fig. 1: Schematic of the cart pole system

TABLE I: Parameters and Specifications

Variable	Value	Unit
m	0.1	kg
M	1	kg
l	0.5	m
I	0.033	$kg \cdot m^2$
b	0.0	$kg \cdot m/s^2$
g	9.81	m/s^2

System Modeling

2. MPC Controller Implementation

- Implemented using pyMPC library
- Utilized the OSQP solver
- Prediction horizon set to 20
- Controller parameters tuned manually

3. MPC + RL Residual Controller Architecture

- $u_{total} = u_{MPC} + u_{residual}$
- Role of residual RL agent (expected)
  - Compensates for model mismatch
  - Improves disturbance rejection and overall control performance

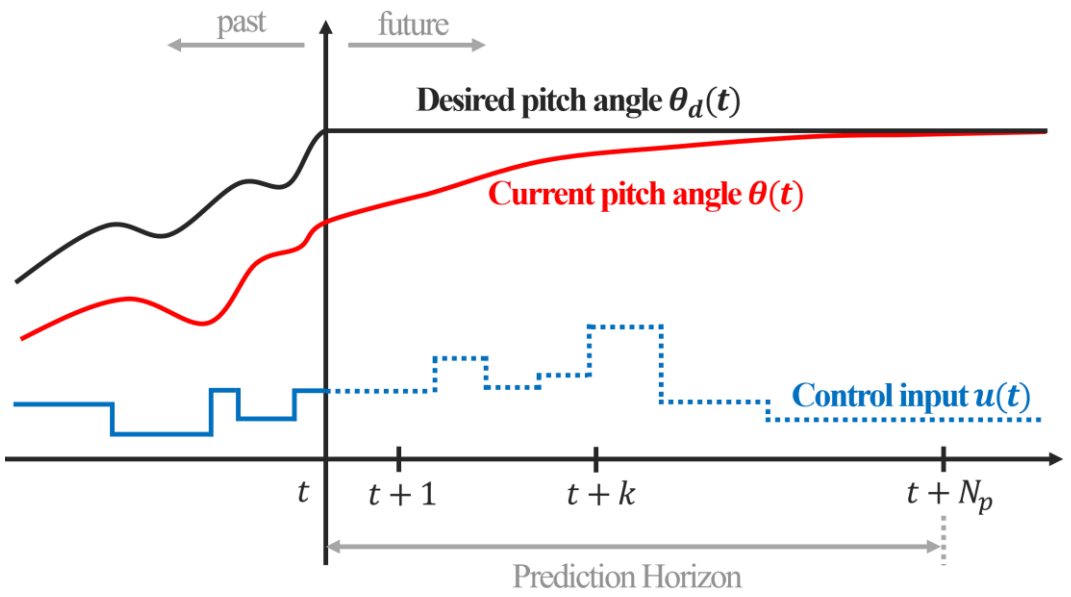


Fig. 2: Illustration of the MPC prediction horizon, showing future reference tracking and the corresponding optimized control inputs.

Environment Configuration

1. Algorithm & hyperparameters

- Algorithm : PPO
- n_steps: 2048
- Batch size: 34
- N_epochs: 10
- Gamma: 0.99
- Learning rate: 10e-4
- Clip_range: 0.2
- Total_timesteps: 600,000

2. Observation, Action space

- Observation: $\{x, \dot{x}, \theta, \dot{\theta}\}$
- Action: scalar residual force [-1, 1] → scaled

3. Reward function

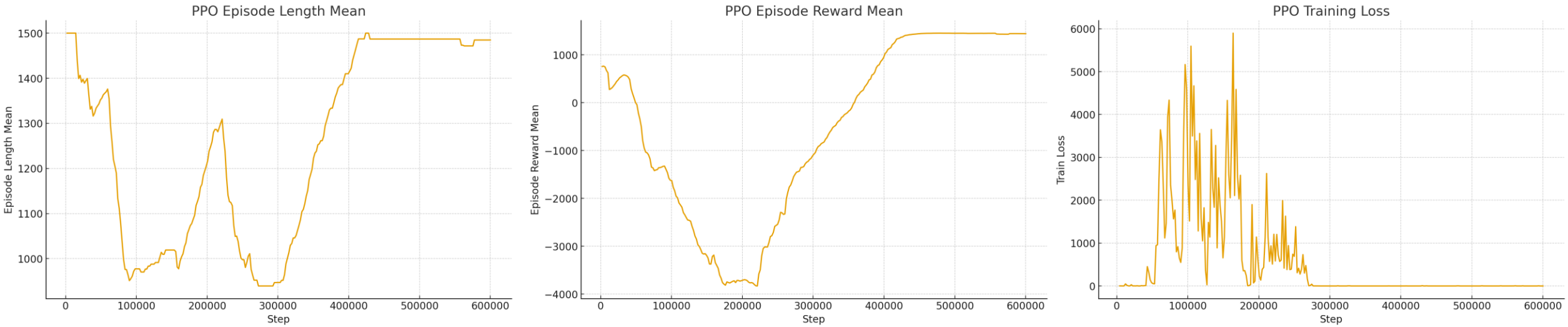
- Alive reward : +1
- Angle penalty : $-5 \times \theta^2$
- Position penalty : $-2 \times \theta^2$
- Residual magnitude penalty: $0.001 \times u_{residual}^2$

4. Domain Randomization

- Alive reward : +1
- Pole mass randomization
  - Pole mass randomized 0.05 kg ~ 1.0 kg
- Random push
  - Random push time: 0.3 ~ 1.2 sec
  - Random push size
    - $\dot{\theta}$: -2 ~ 2 rad/s
    - $\dot{x}$: -0.8 ~ 0.8 m/s
- Randomly pushed 70% of the time

Experiment Results

Training Results



(a) (b) (c)

Fig. 3: Training results of the MPC+RL agent using PPO. (a) Mean episode length. (b) Mean episode reward. (c) Training Loss

Training Hardware & Software

- OS: Windows 11
- CUDA 13.0
- pytorch: 2.9.1
- Graphics card: RTX 5070 TI

Experiment Results

Experiment A: Pole Mass Mismatch

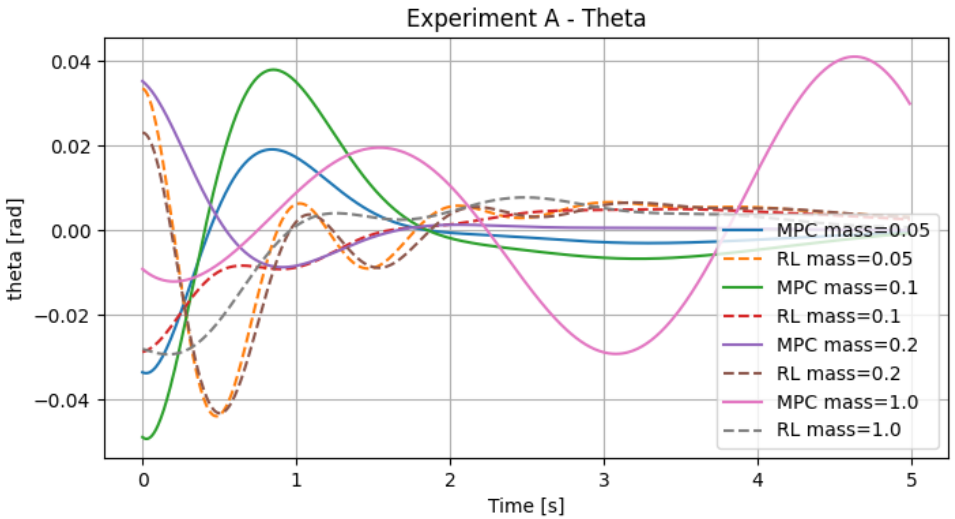


Fig. 4: θ trajectories for different masses in Experiment A

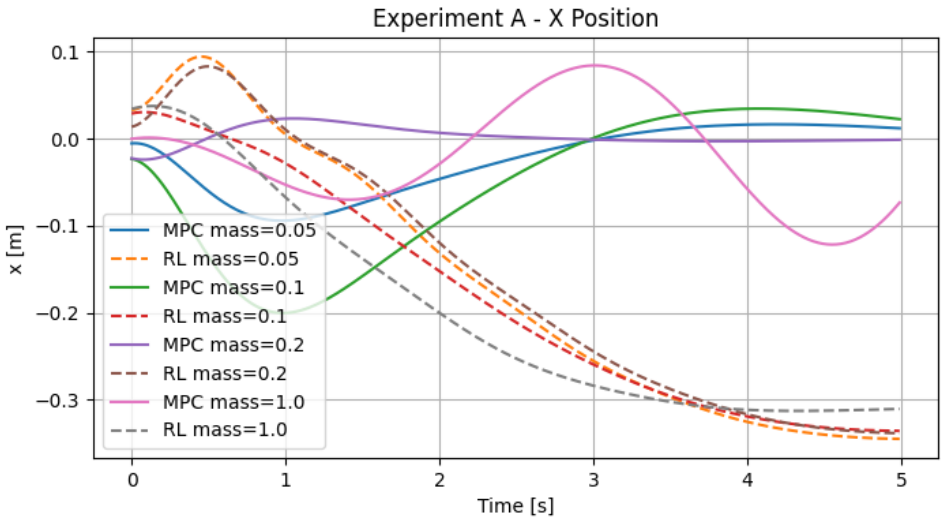


Fig. 5: x position for different masses in Experiment A

Pole mass mismatch

- Nominal pole mass : 0.1 kg
- Mass term used in MPC: [0.05, 0.1, 0.2, 1.0]
- Resets at a random pole angle and position ± 0.05

Results

- Both MPC and MPC+RL performs well for small moderate mass mismatches (0.05 ~ 0.2 kg)
- When pole mass is 1.0 kg, θ starts to diverge (see Fig. 3 - MPC mass=1.0)

Experiment Results

Experiment B: Push disturbance

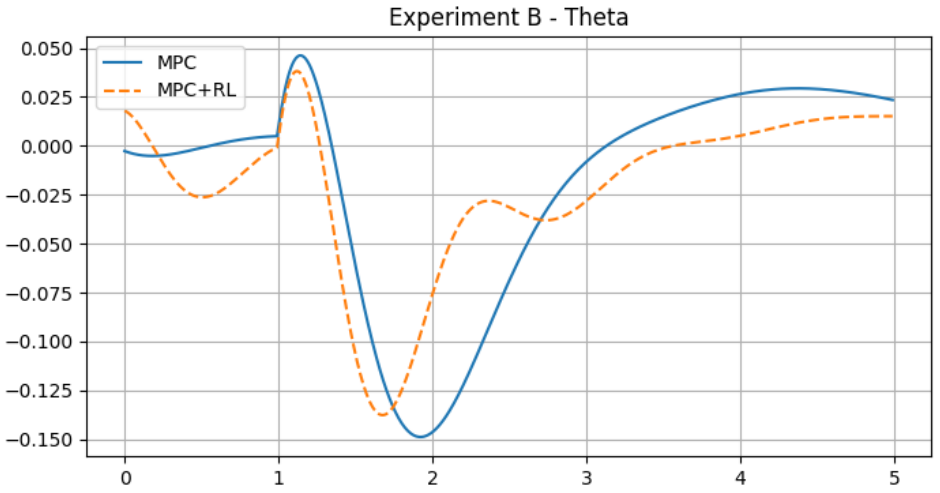


Fig. 6: θ trajectory under push disturbance at nominal mass (m=0.1 kg)

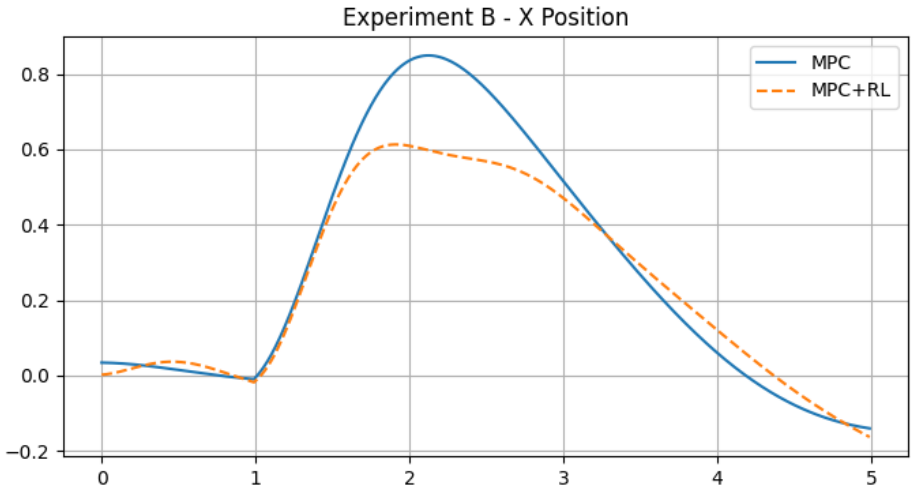


Fig. 7: x position trajectory under push disturbance at nominal mass (m=0.1 kg)

Push disturbance

- Nominal pole mass : 0.1 kg
- Push : $\dot{x}$= 0.5 m/s, $\dot{\theta}$=0.5 rad/s
- Resets at a random pole angle and position ± 0.05

Results

- Fig. 5 and Fig. 6 show that the MPC+RL model has a smaller overshoot compared to the MPC only model.
- The MPC+RL model shows less oscillatory behavior under push disturbance

Experiment Results

Experiment C: Mass mismatch + Push disturbance

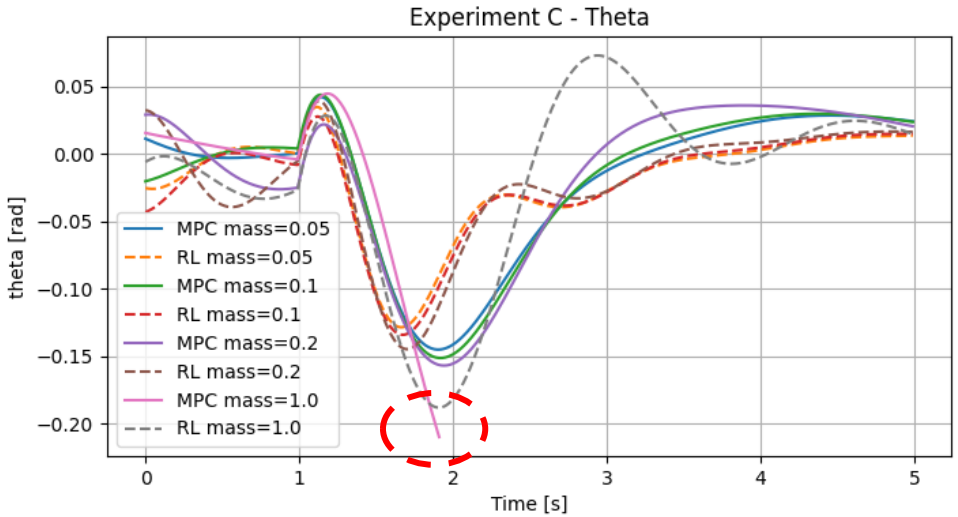


Fig. 8: θ trajectory under mass mismatch and push disturbance

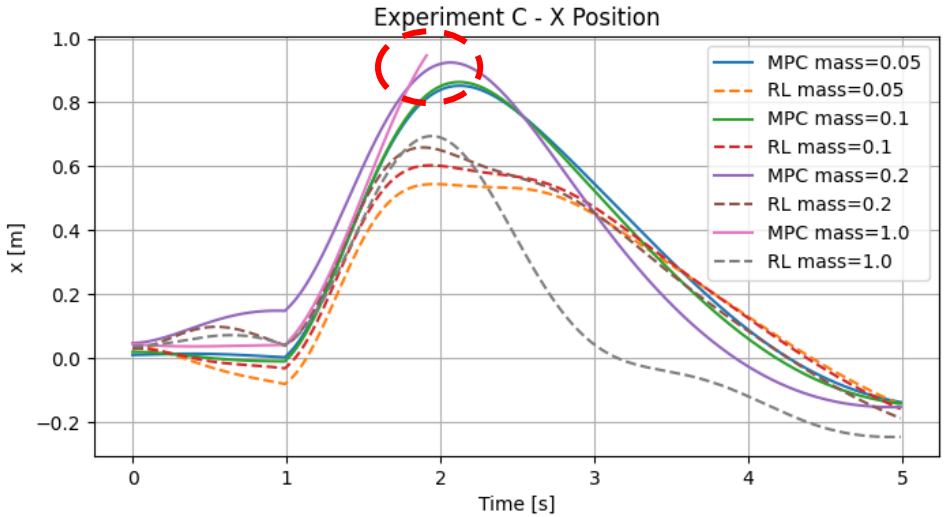


Fig. 9: x position trajectory under mass mismatch and push disturbance

Mass mismatch + Push disturbance

- Nominal pole mass : 0.1 kg
- Mass term used in MPC: [0.05, 0.1, 0.2, 1.0]
- Push : $\dot{x}$= 0.5 m/s, $\dot{\theta}$=0.5 rad/s
- Resets at a random pole angle and position $\pm$0.05

Results

- Fig. 7 and Fig. 8 show that the MPC+RL model generally has a smaller overshoot compared to the MPC only model.
- Model **MPC mass=1.0 diverges** while **MPC+RL mass=1.0 converges**

Comparison with SAC

Comparing MPC+RL policy, trained using PPO VS SAC

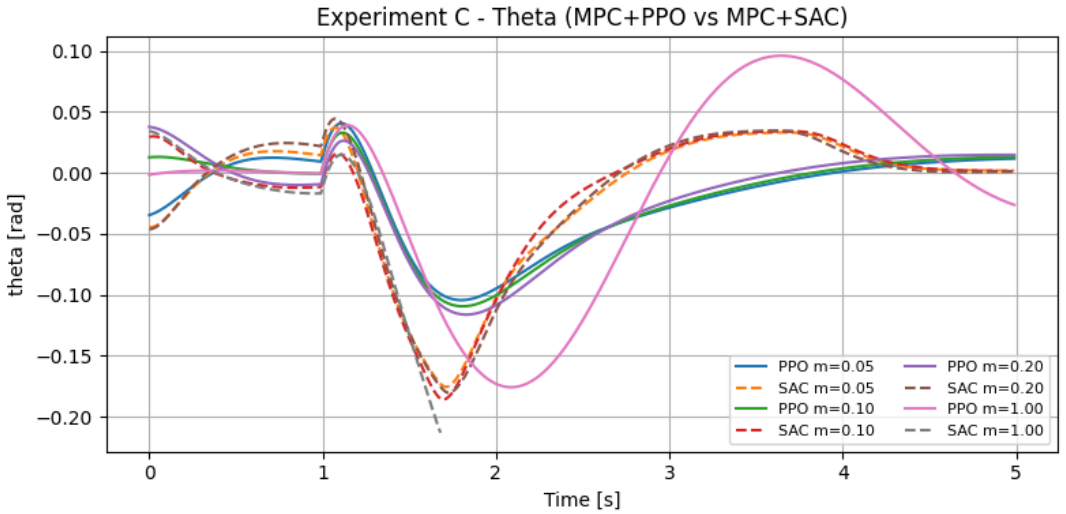


Fig. 10: θ trajectory under mass mismatch and push disturbance comparing PPO and SAC

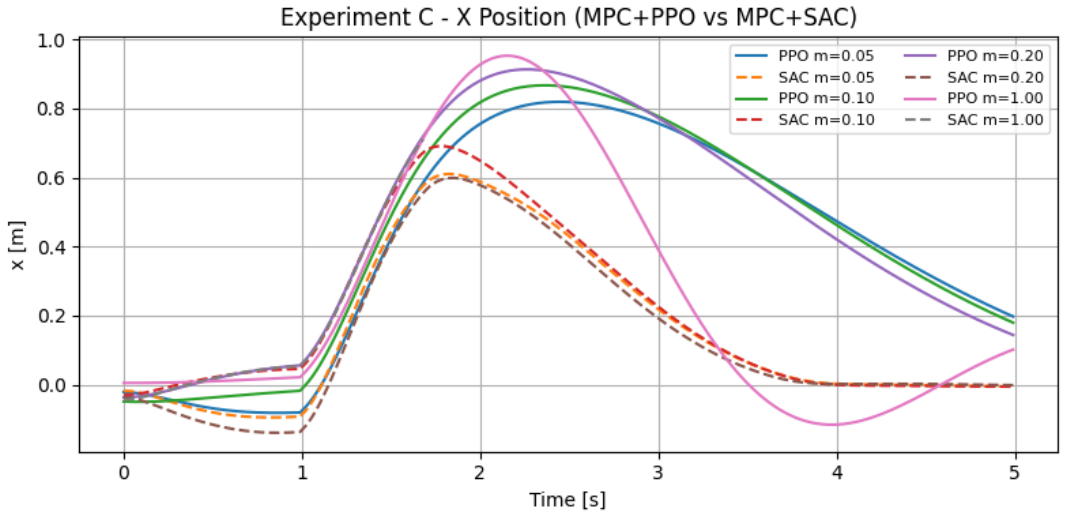


Fig. 11: x position trajectory under mass mismatch and push disturbance comparing PPO and SAC

PPO VS SAC

Total timesteps

PPO: 600k, SAC: 100k

To keep the total training duration comparable between algorithms, SAC was run for fewer steps.

Results

- As can be seen in Fig. 10 and Fig. 11, PPO and SAC performed similarly
- In some cases, the response time is slightly better for SAC

Results & Conclusion

<Results>

- Both MPC and MPC+RL controllers perform similarly under small model mismatches
- However, when model mismatch is large, the MPC+RL controller performs noticeably better than MPC only
- Specifically, MPC+RL models seem to have smaller overshoot while converging faster than MPC only
- MPC+RL models have less oscillatory behaviors under push disturbances
- Experiment B shows that compared to a well tuned MPC controller, MPC+RL controller is more robust under push disturbances
- Comparison between the algorithms trained by PPO and SAC showed similar results between the two training methods
- SAC required significantly more training time compared to PPO

<Conclusion>

- Under mild model mismatch and no disturbances, MPC performs as well as MPC+RL
- Under large model mismatch without disturbances, MPC fails while MPC+RL maintains partial stability
- Overall, the combined controller demonstrates clear benefits in robustness when the system experiences parameter uncertainty or unmodeled nonlinear effects

References

I. C. E. García, D. M. Prett, and M. Morari, ″Model predictive control: Theory and practice — a survey,″ Automatica, vol. 25, no. 3, pp. 335–348, May 1989.

II. A. Bemporad and M. Morari, ″Robust model predictive control: A survey,″ in Robustness in Identification and Control, A. Garulli, A. Tesi, and A. Vicino, Eds., Lecture Notes in Control and Information Sciences, vol. 245, Springer-Verlag, London, 1999, pp. 207–226. doi:10.1007/BFb0109870

III. E. Camacho and C. Bordons, Model Predictive Control, 2nd ed., London, UK: Springer, 2007.

IV. M. Nikolaou, ″Model predictive controllers: A critical synthesis of theory and industrial needs,″ Advances in Chemical Engineering, vol. 26, Academic Press, 2001, pp. 131–204.

V. A. Bemporad, F. Borrelli, and M. Morari, ″Model predictive control based on linear programming — The explicit solution,″ IEEE Trans. Autom. Control, vol. 47, no. 12, pp. 1974–1985, 2002.

VI. Forgione, M., Piga, D. and Bemporad, A., 2020. Efficient calibration of embedded MPC. IFAC-PapersOnLine, 53(2), pp.5189-5194.